

Flow-Pattern Anomaly Detection at the IoT Gateway

A pre-trained Mamba detector for known and zero-day attacks on smart-home devices

Madhav Dogra • Final Project Proposal

SNAPSHOT

A detector that sits at the home gateway and inspects every flow to or from a smart-home device. **Known attacks** are caught by a supervised classifier; **zero-day attacks** by a one-class anomaly scorer. Both heads ride a single **Mamba** encoder pre-trained on benign traffic. The detector runs in two operating modes — a low-power **default** that runs continuously, and an admin-switched **strong** mode with the full pipeline and SHAP attributions. New attack types are absorbed by re-training only the lightweight classifier head, not the encoder. Target: <10 ms p95 per flow on Raspberry-Pi-class hardware after int8 quantisation.

1 The setting

The target environment is the modern smart home. The device population includes **IP cameras, motion and environmental sensors, smart bulbs, voice assistants, thermostats, video doorbells, smart plugs, and the gateway itself**. CICIoT2023, the primary training dataset, is dominated by exactly this device class: 105 real consumer IoT devices, 33 attack types, 7 attack categories.

1.1 Why these devices are the soft layer

They ship with default credentials, run firmware that is rarely updated, lack the compute to host any endpoint security agent, and are often physically accessible. The Mirai botnet — built almost entirely from compromised IP cameras and home routers — is the canonical demonstration that this surface is exploitable at scale. In a defence context the stakes rise: field sensors, surveillance equipment and base infrastructure are high-value, low-trust devices that cannot be hardened one by one.

1.2 Why the gateway is the right vantage point

Putting software *on* the devices is not possible. Watching them on the network is. The gateway is the single point through which every device flow passes on its way upstream, and the only place where modern compute can sit without instrumenting the device fleet. The trade-off: gateway hardware is small (Raspberry-Pi-class, single-digit watts), so the detector must be lightweight. This one constraint shapes every architectural decision that follows.

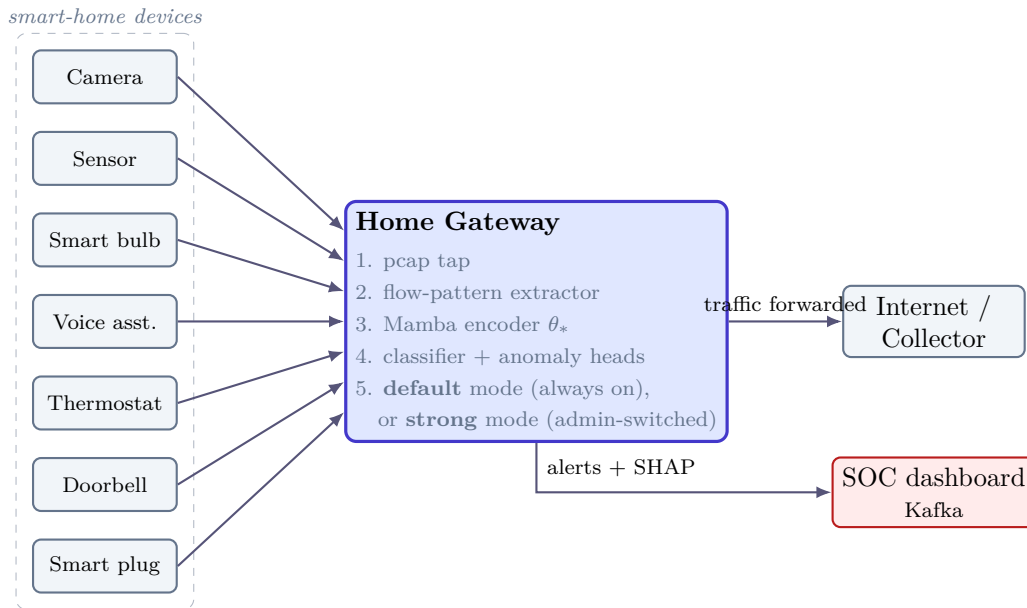


Figure 1: *

Figure 1 — The detector lives inside the home gateway. Device traffic continues upstream unchanged; alerts and SHAP attributions are sent out-of-band to a SOC dashboard.

2 What the detector must do

Four properties, in priority order, plus one hard constraint.

1. **Catch known attacks** with high precision — DDoS, reconnaissance, brute force, Mirai-style botnet traffic. The supervised classifier owns this.
2. **Catch unknown (zero-day) attacks.** A detector that only recognises known patterns misses the next new thing. The one-class anomaly scorer owns this: it learns “normal” and flags whatever does not fit, without ever training on the attack.
3. **Work on encrypted traffic.** Payloads are opaque, so the detector reads flow metadata only — packet sizes, directions, timings, flag distributions, header fields.
4. **Explain every alert.** “The model said so” is not actionable. Every flagged flow carries an attribution of which features drove the decision, so an analyst can act on it.

The constraint: all of this runs at the gateway, on small hardware, in under 10 ms per flow. Existing approaches each cover one or two of these. Signature firewalls catch known attacks but miss zero-days. Enterprise IDS is tuned for human traffic and false-alarms on periodic machine-to-machine patterns. Transformer detectors are accurate but do not fit on edge hardware. Most academic work benchmarks on a single dataset and never measures evasion. The gap is a detector that does all four *and* is stress-tested.

3 The input representation

Conventional IoT IDS collapses each flow into a vector of ~ 80 summary statistics (mean packet size, IAT standard deviation, flag counts, ...) before the model sees it. This is convenient but *permutation-invariant* — shuffle the packets and the statistics do not change — which destroys the temporal structure that attack signatures live in.

KEY IDEA

Keep the order. A flow is the first $K \approx 32$ packets of the connection, kept as an ordered sequence $\mathbf{x} = (p_1, \dots, p_K)$. Each p_t is a small per-packet feature vector — size, direction, inter-arrival time, TCP flags, header fields. A sequence model can then learn what a normal sequence looks like, and what a malicious one looks like.

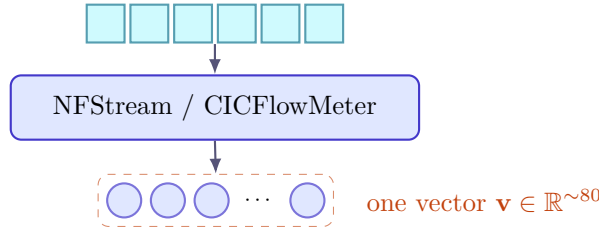
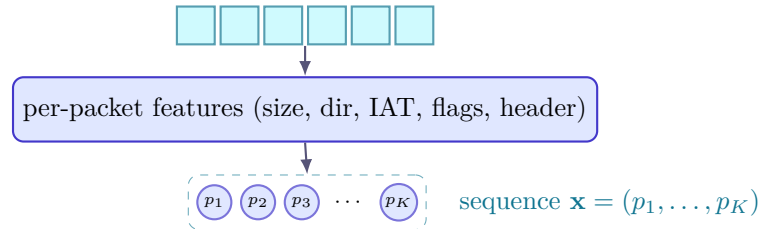
Conventional — aggregated (order lost)**Ours — flow pattern (order preserved)**

Figure 2: *

Figure 2 — Two ways to feed a flow to a model. Aggregation collapses everything into one vector and drops ordering; the flow-pattern keeps the packets in order so a sequence model can use the temporal structure.

3.1 Preprocessing — transforms before the model

Traffic features are heavy-tailed: a handful of flows carry megabytes while most carry a few hundred bytes; a few connections idle for seconds while most are sub-millisecond. Raw, this long right tail dominates every distance computation and swamps clustering and one-class scoring.

KEY IDEA

Apply $\log(1 + x)$ to the heavy-tailed per-packet and per-flow features. It compresses the tail, makes each distribution near-symmetric, and tightens the class clusters. Standardise (zero-mean, unit-variance) *after* the log. The **Yeo–Johnson** power transform generalises this — it learns the best exponent per feature and handles zeros and negatives automatically (one `scikit-learn` call); $\log(1 + x)$ is the transparent fallback.

WATCH OUT

log and exp move data in *opposite* directions. For positive, right-skewed traffic features, log compresses large values toward the bulk (good); e^x expands them and stretches the tail further (worse). The compacting transform here is the logarithm, not the exponential.

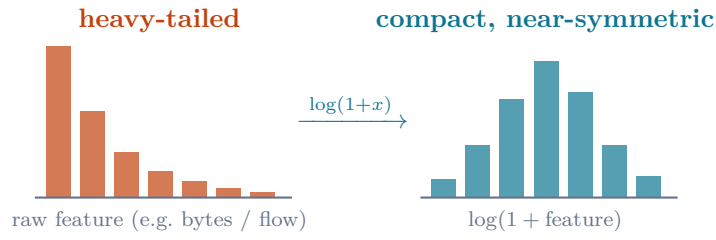


Figure 3: *

Figure 3 — The log transform on a heavy-tailed traffic feature. The long right tail (amber) is pulled into a compact, roughly symmetric distribution (teal), so distances and one-class scores reflect the bulk of the data rather than a few outliers.

4 Feature significance and visualisation

Two questions get conflated and should be kept apart: *which input dimensions carry class signal*, and *do the classes separate in the chosen representation*. They need different tools.

KEY IDEA

Rank features with **mutual information** and **SHAP**. Use **t-SNE / UMAP** **only** to confirm separability. t-SNE and UMAP are visualisation tools, not feature rankers: their two output axes are nonlinear blends of *all* the inputs, with no per-feature loadings to read off. Reading a feature ranking off a t-SNE plot is the mistake to avoid.

Mutual information is the univariate, model-free relevance score and is the feature-selection step in the pipeline; it cuts dimensionality by more than half with negligible accuracy loss. SHAP, already used for per-alert explanation, doubles as a global importance ranking by averaging $|\text{attribution}|$ across flows. Permutation importance is a cheap cross-check. t-SNE/UMAP then answer one question only: do benign traffic and each attack category fall into distinct clusters. Two cautions: cluster *sizes* and the *distances between* clusters in a t-SNE plot are not meaningful (only local neighbourhoods are preserved), and the picture changes with perplexity. It is a qualitative diagnostic, never a measurement.

Hand-selecting input dimensions applies to the classical baselines (XGBoost / Isolation Forest on the aggregated statistics). The Mamba arm consumes the ordered sequence and learns its own representation $\mathbf{z}$; for that arm the high-value use of t-SNE/UMAP is to visualise $\mathbf{z}$ coloured by class, confirming the encoder built a separable space — the precondition the anomaly head depends on.

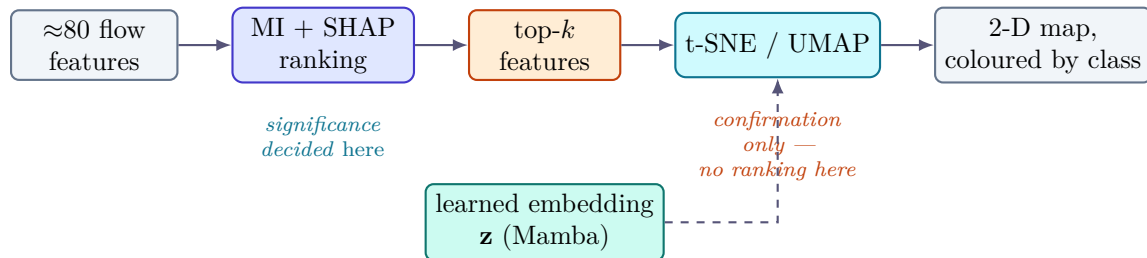


Figure 4: *

Figure 4 — Rank, then visualise. Feature significance is produced by mutual information and SHAP (indigo); t-SNE/UMAP (teal) only confirms that the kept features separate the classes. For the Mamba arm the same plot is applied to the learned embedding $\mathbf{z}$ (dashed branch).

5 The architecture

A single **Mamba encoder** processes the sequence $\mathbf{x}$ and produces a fixed-size embedding $\mathbf{z}$. Two heads read $\mathbf{z}$ in parallel:

- **Classifier head** — a small MLP $\rightarrow$ softmax over the 7 known attack categories from CICIoT2023. Trained with focal loss to handle the heavy class imbalance.
- **Anomaly head** — a one-class deep SVDD scorer trained *only on benign embeddings*. It does not know about any specific attack; it measures distance from “normal.”

KEY IDEA

The anomaly head reads a **projected embedding** $\mathbf{z}_{\text{ano}} = g_{\phi}(\mathbf{z})$ from a small learnable MLP ($\mathbb{R}^{128} \rightarrow \mathbb{R}^{32}$). One-class scorers suffer the curse of dimensionality: in high dimensions, distances concentrate and lose discriminative power. Projecting to a smaller anomaly space sharpens the score. The projection is trained jointly with the SVDD objective.

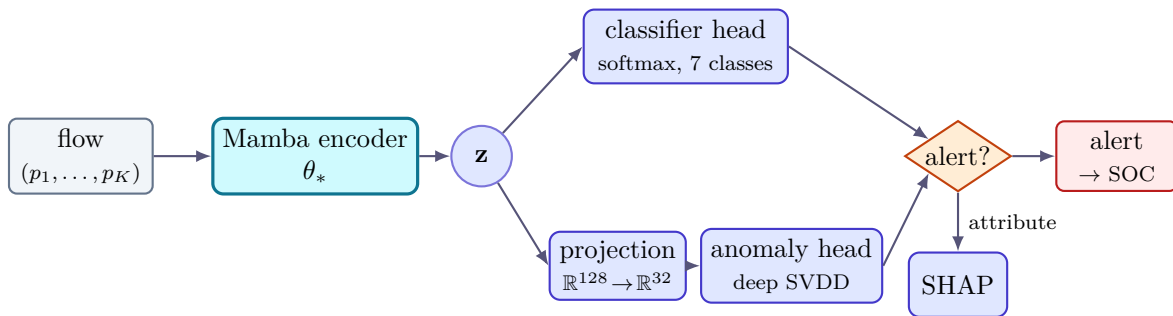


Figure 5: *

Figure 5 — One shared encoder, two heads. The classifier scores known categories; the projected anomaly head scores distance from normal. If either crosses its threshold, an alert is raised, SHAP attributes it, and it is forwarded to the SOC.

5.1 Why Mamba and not a transformer

The reference architecture for pre-trained encrypted-traffic detectors is ET-BERT, a BERT-base-scale ($\sim 110\text{M}$ parameter) transformer. It works but does not fit on a Raspberry Pi. NetMamba (2024) showed that a Mamba state-space model reaches ET-BERT-class accuracy at substantially lower parameter and latency cost, because Mamba runs in linear time in sequence length and has much lower per-token overhead than attention.

5.2 Why a shared encoder

Pre-training is the expensive part; sharing the encoder pays it once. It also gives the anomaly head a representation of “normal” learned from a large benign corpus — exactly what a one-class scorer needs.

6 The anomaly sphere

The anomaly head formalises “inside the sphere is normal, outside is not.”

KEY IDEA

Deep SVDD learns a mapping that packs benign embeddings into a minimum-volume hypersphere around a centre c . The anomaly score is the distance to c ; a flow beyond radius R — calibrated at the 99th benign percentile (1% FPR target) — is flagged. The compactness is the *training objective itself*, learned jointly with the projection. A scalar log/exp cannot carve this boundary; the learned transform can.

An alternative variant is **L2 normalisation**: dividing each embedding by its norm puts every point on the unit sphere’s surface, and “normal vs anomalous” becomes an angular decision — a spherical cap (cosine similarity to a benign prototype above a threshold) is in-class. This is the contrastive-learning flavour of the same idea and a clean A/B against radial deep SVDD. Note that normalising alone puts *everything* on the sphere; a learned threshold (the cap) still separates in- from out-of-class.

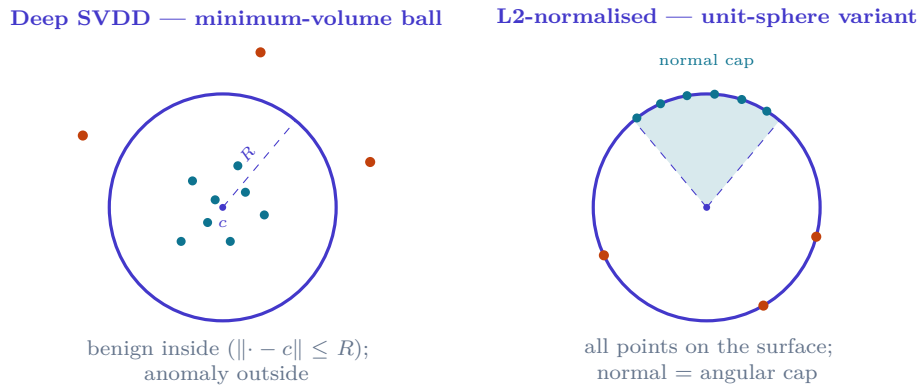


Figure 6: *

Figure 6 — Two ways to realise “inside = in-class.” Left: deep SVDD packs benign embeddings (teal) into a minimum-volume ball of radius R around c ; anomalies (amber) fall outside. Right: L2 normalisation places every point on the unit sphere’s surface and “normal” becomes a learned angular cap. Both boundaries are learned from benign data.

7 Two operating modes — default and strong

The gateway runs in one of two configurations. The choice is an **operator-level setting** flipped by the admin, not a per-flow decision.

- **Default mode — low power, always on.** Smaller Mamba, smaller K (e.g. $K = 8$), *only the anomaly head* active, no SHAP. Detection is on, the compute cost is small enough for continuous use, and both known and novel attacks are caught via the anomaly score alone.
- **Strong mode — admin-switched, full pipeline.** Full Mamba, $K = 32$, *both* heads (anomaly via the projection MLP), plus SHAP on every alert. Switched on for an elevated threat level, an active investigation, a high-value time window, or simply maximum detection fidelity.

The same artifacts — encoder θ_* , classifier, projection, anomaly head — serve both modes; what differs is which components are engaged and at what size. Both modes alert to the SOC; strong mode additionally produces per-packet SHAP attributions.

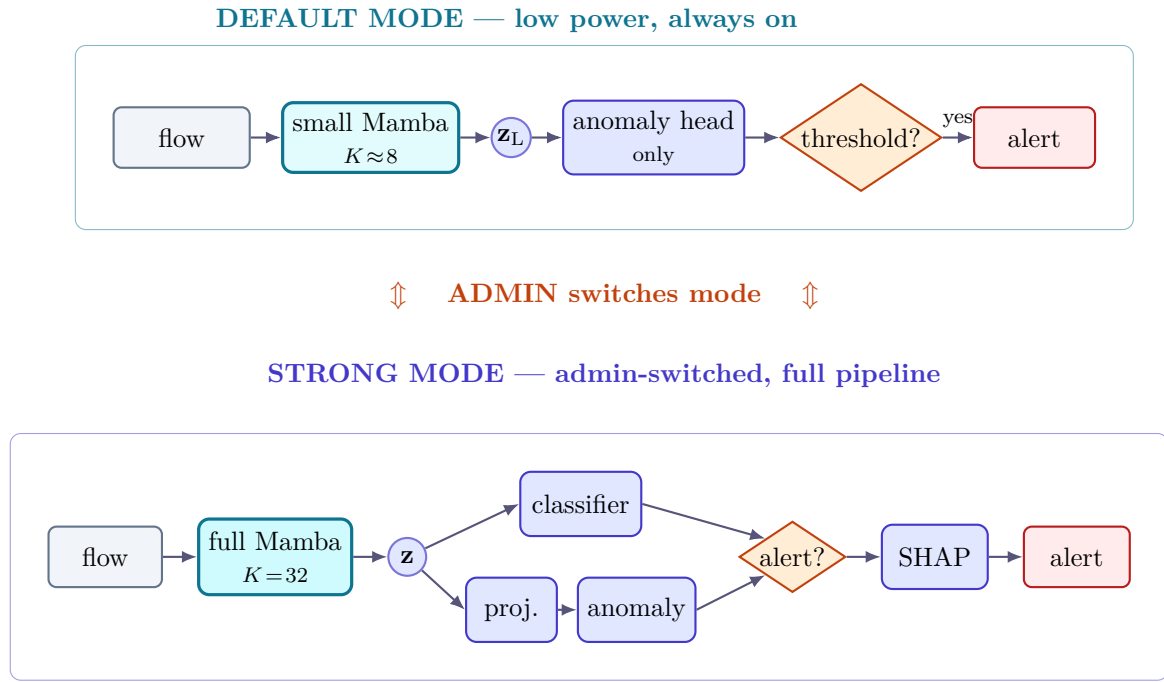


Figure 7: *

Figure 7 — The two operating modes. Default runs continuously on a small Mamba with only the anomaly head. Strong is switched on by the admin and engages the full pipeline: full Mamba, classifier, projected anomaly head, and SHAP. The admin chooses which mode is active; the gateway then runs entirely in that mode until the next switch.

8 Training — three stages, then a loop

The encoder is built first without labels, specialised with labels, then the anomaly head is fitted on top.

8.1 Stage A — pre-train the encoder, no labels

Take a large set of **benign** flows. Randomly mask $\sim 15\%$ of packet positions in each sequence and train the Mamba encoder to predict the masked packets from context — the SSL recipe pioneered by ET-BERT, applied to per-packet features. Output: an encoder θ_0 that has learned what normal traffic looks like.

8.2 Stage B — fine-tune with labels

Attach a fresh classification head on θ_0 and jointly fine-tune the encoder and head on labelled CICIoT2023 attack data with focal loss. Output: the fine-tuned encoder θ_* and the trained classifier.

8.3 Stage C — fit the anomaly head

Freeze θ_* . Push a held-out benign set through it to get embeddings, train the projection MLP, and fit deep SVDD on the projected embeddings. Calibrate the threshold at the 99th percentile of benign scores.

8.4 The loop — absorbing new attack types

When an unseen attack appears, the system does *not* need full retraining:

1. The anomaly head flags the novel attack on day zero — it just looks unlike normal.
2. The SOC labels a sample of the flagged flows as a new class.
3. Extend the classifier softmax by one class and fine-tune **only the classifier head**, with a small replay buffer of old classes to prevent forgetting. The encoder stays frozen. Minutes, not days.

4. Optionally recalibrate the anomaly threshold on a fresh benign set.

Full encoder retraining is triggered only if the benign distribution itself shifts (new device types added). “What’s normal” lives in the expensive encoder; “what’s malicious” lives in the cheap heads.

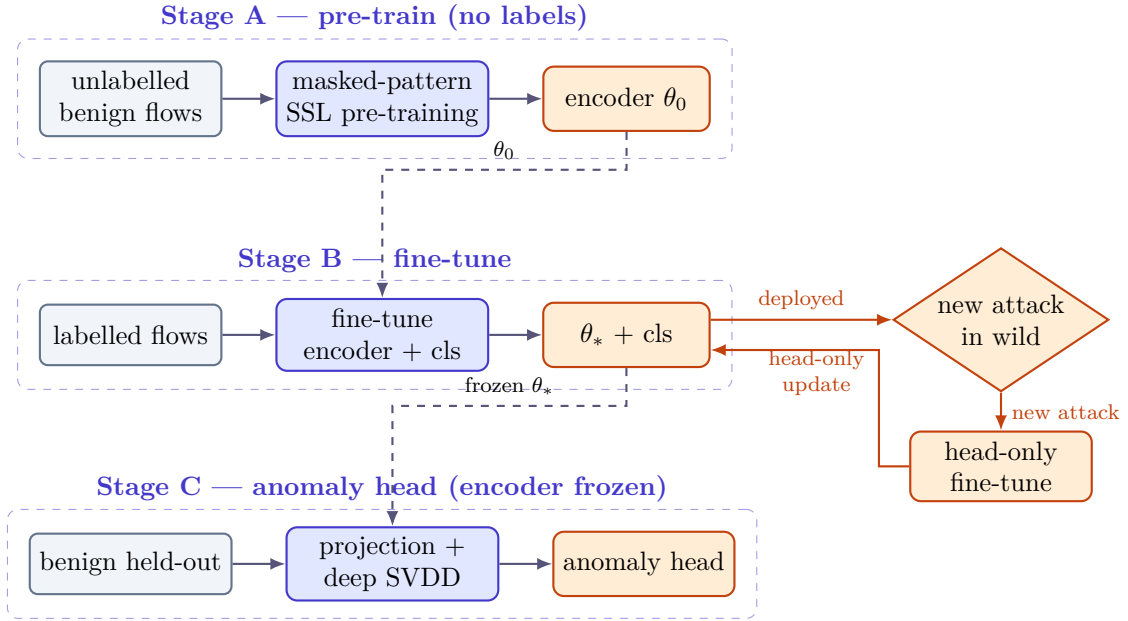


Figure 8: *

Figure 8 — The three training stages produce the deployed detector. The amber loop on the right is the progressive-learning path: a new attack triggers a head-only re-train — the expensive encoder stays put.

8.5 Continual learning — the correlation problem

The hard case is correlation: a new attack class often resembles an existing one (a new DDoS variant looks like the standard DDoS class), so naive head retraining quietly confuses them. Three failure modes appear — **catastrophic forgetting** (the head drifts toward the new class), **probability-mass theft** (the new softmax dimension steals confidence from the correlated class), and **spurious hard labels** (boundary flows forced into one class, hiding the ambiguity from the analyst). Before adding a class, embed the candidate flows through the frozen encoder and inspect where they land.

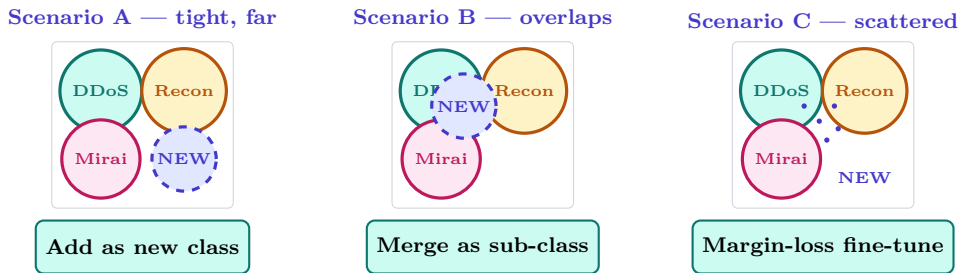


Figure 9: *

Figure 9 — Three embedding-space scenarios for a new attack class (existing classes solid, new class dashed indigo). The geometry dictates strategy: tight & far → add; overlapping → merge as sub-class; scattered between → enforce separation via margin loss. Run before any softmax change is committed.

8.6 The update protocol — four defences, in order

1. **Replay buffer (primary)**. Persist ~100–500 labelled flows per existing class; batch them in on every fine-tuning step so the head still sees every old class. Cheap, well-studied, usually enough.
2. **Knowledge distillation (Learning-without-Forgetting)**. Regularise the new head toward

agreeing with the old head on old-class data — one extra forward pass through the frozen old head.

3. **Embedding-space sanity check.** Locate the new samples relative to existing class prototypes; branch on the three scenarios above.
4. **Surface uncertainty, don't force labels.** If the softmax is split (55% NEW, 40% DDoS), pass both confidences plus the anomaly score and SHAP to the SOC instead of forcing a hard label.

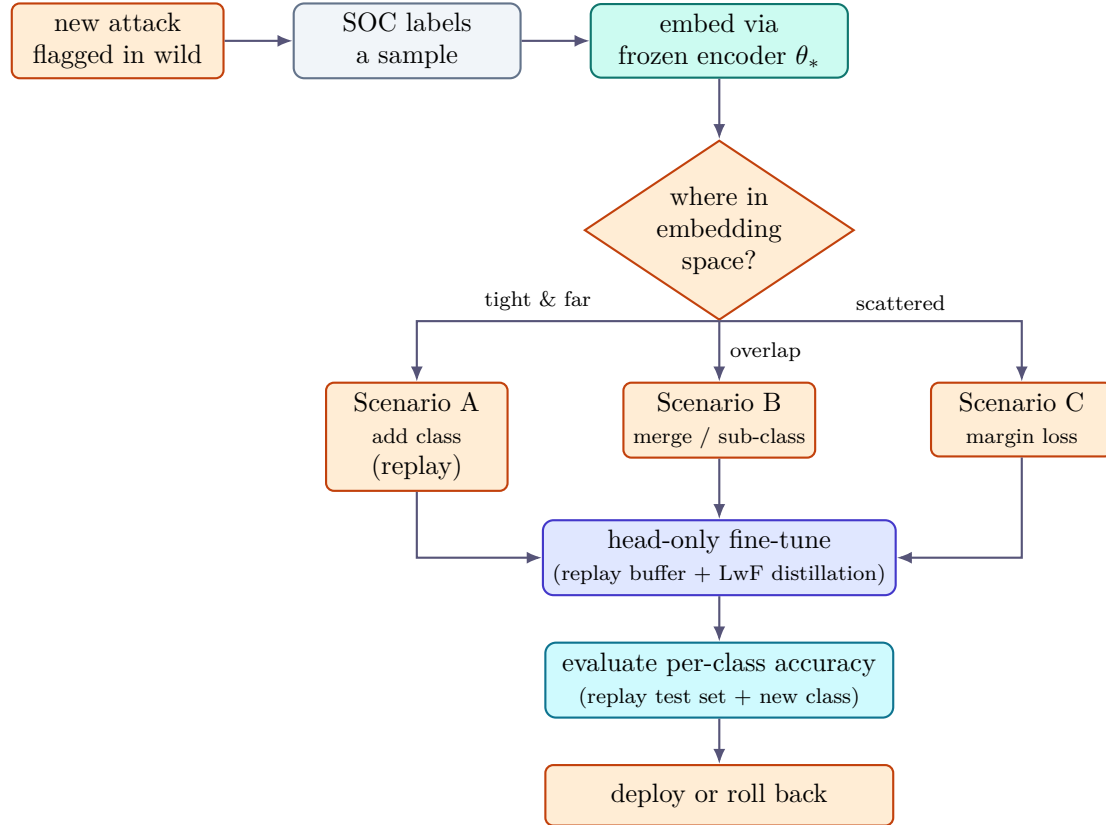


Figure 10: *

Figure 10 — The progressive-learning update protocol. The encoder is never touched. The embedding-space scenario decides add / merge / separate; the head is fine-tuned with replay and distillation; per-class accuracy is checked before deploying. If any existing class regresses beyond a set threshold, the update is rolled back automatically.

9 Deployment and quantisation

The deployed detector must fit on gateway-class hardware. Quantisation — storing weights and activations in fewer bits than fp32 — is the standard route to a smaller, faster model. The question is whether Mamba quantises cleanly, since most published quantisation work targets transformers.

KEY IDEA

Mamba does quantise, but the selective state-space core has activation outliers that compound through its recurrence. The practical strategy is *mixed precision*: int8 for the linear projections (most of the parameters), fp16 for the SSM core (the sensitive part).

A Mamba block has two parts. The **linear projections** (input, gating, output) dominate the parameter count and quantise cleanly to int8. The **selective state-space core** (Δ , the input-dependent B , C , and the recurrent scan) has activation outliers and compounds error along the sequence; int8 cannot represent these without large error. Headline recipe: int8 the projections, fp16

the SSM core — size savings come mostly from the projections anyway.

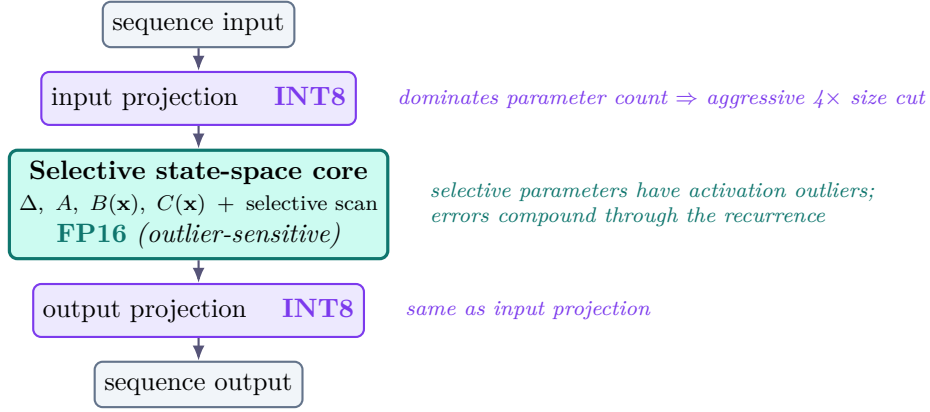


Figure 11: *

Figure 11 — Mixed-precision split inside a Mamba block. Linear projections (violet, int8) carry most of the weights and quantise cleanly; the selective SSM core (teal, fp16) is sensitive and stays at higher precision. This is the practical compromise across the 2024–25 Mamba quantisation literature (Q-Mamba, MambaQuant, PTQ4SSM).

The deployed model is reported in three precisions so the operator can choose:

Configuration	Size	p95 latency	Accuracy hit
fp32 (baseline)	1.0×	1.0×	—
int8 PTQ everywhere	0.25×	~0.5×	risk on SSM core
Mixed precision (recommended)	~ 0.30×	~0.55×	smallest drop

If int8-everywhere degrades accuracy too far, mixed precision is the recommended configuration. Quantisation-aware training is the fallback for the SSM core if even fp16 leaves quality on the table.

10 Evaluation

10.1 Datasets

CICIoT2023 (primary). **TON_IoT** for cross-dataset generalisation — the trained model evaluated on a dataset it has never seen. **IoT-23** for a real-world malware check (Mirai, Torii, Okiru, Gafgyt).

10.2 Baselines — and what each one is for

- **ET-BERT** — the published-SOTA upper bound. Quantifies the accuracy given up by going to Mamba.
- **CNN-BiLSTM on aggregated statistics** — the representation control. Isolates the gain from the flow-pattern input.
- **XGBoost on aggregated statistics** — the classical-floor challenge. Gradient-boosted trees on hand-crafted features are routinely competitive in IDS literature. The deep detector earns its complexity only if it beats XGBoost on what XGBoost cannot target: **zero-day detection** (XGBoost is supervised-only), **cross-dataset transfer**, and **adversarial robustness**. If XGBoost matches in-distribution, that is reported as a finding.
- **Isolation Forest** — the classical unsupervised floor for the anomaly head.

10.3 Metrics

Per-class precision / recall / F1. PR-AUC (the honest metric under heavy class imbalance). Inference latency p50 / p95 / p99 per flow. Model size in MB after int8 quantisation.

10.4 Adversarial study

Fast Gradient Sign Method (FGSM) on the per-packet sequence, perturbations clipped to valid feature ranges. Detection rate vs. perturbation magnitude ε , plotted for both heads. Secondary check: SHAP attribution patterns on adversarial inputs tend to differ from clean inputs — a useful fallback signal.

10.5 Zero-day proxy

Leave-one-class-out: hide an attack class from the supervised head during Stage B, then measure how well the anomaly head detects it at test time. Repeat per class. This is the concrete answer to “how do you evaluate something you haven’t trained on.”

11 Plan, deliverables, risks

11.1 Twelve-week plan

Weeks 1–2 setup and dataset validation; week 3 flow-pattern extractor, preprocessing transforms, and classical baselines; weeks 4–5 Stage A pre-training; week 6 Stage B fine-tuning and CNN-BiLSTM + ET-BERT comparison; week 7 Stage C anomaly head and leave-one-class-out; week 8 TON_IoT + IoT-23 evaluation; week 9 SHAP integration and t-SNE/UMAP embedding analysis; week 10 FGSM adversarial study; week 11 int8 quantisation and gateway latency benchmark; week 12 stretch goal, final report, presentation.

11.2 Deliverables

A reproducible code repository covering the full pipeline. The pre-trained Mamba encoder checkpoint θ_* released with the report. A benchmark table covering Mamba vs. ET-BERT vs. CNN-BiLSTM vs. XGBoost vs. Isolation Forest across CICIoT2023, TON_IoT, IoT-23. FGSM degradation curves. Latency / memory profile on gateway hardware. Written report.

11.3 Risks — and what I do about them

WATCH OUT

- **Pre-training compute blows budget.** Warm-start from a public Mamba checkpoint and adapt on benign IoT traffic rather than train from scratch.
- **Class imbalance hides minority-class failure.** Focal loss + always report per-class metrics.
- **Cross-dataset accuracy drop.** Measure honestly; this is the standard caveat in IDS literature, not a failure mode to hide.
- **Edge memory overrun.** Int8 quantise, fall back to a smaller Mamba, reduce K as a last resort.
- **Adversarial sensitivity.** The FGSM study quantifies it; the SHAP fingerprint is a secondary defence.

11.4 Stretch goals

- Swap Mamba → Mamba-2 (Structured State Space Duality) if latency headroom allows.
- Add an E-GraphSAGE branch over a per-window device-communication graph.
- Federated pre-training across multiple homes / sites.

Key references

- NetMamba, Wang et al. 2024 — arxiv.org/abs/2405.11449
- ET-BERT, Lin et al. WWW 2022 — arxiv.org/abs/2202.06335
- Mamba, Gu & Dao 2023 — arxiv.org/abs/2312.00752
- Mamba-2 / SSD, Dao & Gu 2024 — arxiv.org/abs/2405.21060

- **CICIoT2023**, Neto et al. 2023 — mdpi.com/1424-8220/23/13/5941
- **Deep SVDD**, Ruff et al. ICML 2018 — the one-class anomaly scorer.
- **SHAP**, Lundberg & Lee, NeurIPS 2017 — attributions that double as feature importance.
- **t-SNE**, van der Maaten & Hinton, JMLR 2008; **UMAP**, McInnes et al. 2018 — arxiv.org/abs/1802.03426
- **E-GraphSAGE**, Lo et al. — arxiv.org/abs/2103.16329